TP Unidad 7: NoSQL y nuevas formas de almacenamiento

Food Store | Modelo documental, migracion y consultas

Integrantes: Juan Martinez e Isaias Morinigo Repositorio: EIIsuss/TP-NoSQL-BD-II

1. Modelo y migracion

La base PostgreSQL foodstore contiene las tablas pedido, detalle_pedido, cliente y producto. Se migraron los cinco pedidos activos con IDs 1-5 y sus 17 lineas. Cada documento MongoDB conserva el ID, fecha, forma de pago y estado; embebe el ID del cliente y sus lineas con IDs de detalle/producto, nombre del producto, cantidad y precio unitario historico. El precio se guarda como Decimal128 para mantener exactitud monetaria.

Para respetar la consigna de no incluir datos personales reales, la copia MongoDB conserva el id_cliente y usa una etiqueta anonimizada; no se exportaron nombre, correo ni telefono. BSON Date tiene precision de milisegundos, por lo que fecha_hora_origen conserva ademas el timestamp PostgreSQL completo con microsegundos y zona horaria.

Elemento	Diseno / regla
Pedido	_id conserva id_pedido; fecha_hora, fecha_hora_origen, forma_pago, estado.
Cliente	Objeto embebido con id_cliente y nombre anonimizado.
Lineas	Arreglo productos con id_detalle_pedido, id_producto, nombre, cantidad y precio_unitario historico.
Integridad	Mantener FK pedido-cliente y pedido-producto; cantidad > 0; precio >= 0; producto unico por pedido. Mongo requiere controles de aplicacion/validacion.

El script usa upsert para repetirse sin borrar la coleccion. PostgreSQL se consulto en modo de solo lectura; las salidas se guardaron sin datos personales.

2. Consultas ejecutadas

Consulta MongoDB	Resultado
Listado y estado	5 pedidos: IDs 1, 2, 3, 4, 5; todos PENDIENTE.
Filtro estado PENDIENTE	5 pedidos: IDs 1, 2, 3, 4, 5.
Producto id_producto 3485	Pedido 1.
Total > $250.000	Pedido 4; total $302.935,07.

3. Verificacion contra PostgreSQL

PostgreSQL local: base foodstore, version 18.4; MongoDB: version 9.0.4 temporal. Se comparo el mismo subconjunto por ID. Un cotejo automatizado confirmo coincidencia exacta de los cinco encabezados y 17 lineas: timestamp original, forma de pago, estado, id_cliente, IDs/nombres de producto, cantidades y precios. El total se calcula con cantidad por precio historico.

Filtro	PostgreSQL	MongoDB	IDs iguales
Estado PENDIENTE	5: 1, 2, 3, 4, 5	5: 1, 2, 3, 4, 5	Si
Producto 3485	1: 1	1: 1	Si
Total > $250.000	1: 4 ($302.935,07)	1: 4 ($302.935,07)	Si

Totales PostgreSQL por pedido

Pedido	Total
1	$177.083,84
2	$201.351,43
3	$84.519,11
4	$302.935,07
5	$100.255,60

El umbral $250.000 devuelve solo el pedido 4. Evidencias completas sin datos personales: evidencias/evidencia_postgresql.txt, evidencias/evidencia_mongodb.txt y evidencias/comparacion_integridad.txt.

4. Evaluacion con IA

Prompt: Compara PostgreSQL con MongoDB para Food Store. El modelo actual es relacional y separa pedido, detalle_pedido, cliente y producto. Se consulta con frecuencia el pedido completo (cabecera, cliente y lineas). Hay que conservar la coherencia de cantidades y precios historicos. Explica ventajas y riesgos y recomienda mantener PostgreSQL, usar MongoDB para este subconjunto o combinar ambos, justificando con criterios tecnicos.

Sintesis: PostgreSQL es adecuado para el modelo relacional y sus restricciones de integridad; MongoDB permite embeber cliente y lineas para leer el pedido completo en una operacion. Se recomienda mantener PostgreSQL como fuente autoritativa y considerar MongoDB como proyeccion de lectura si la necesidad justifica sincronizar datos.

5. Contraste y conclusion

Afirmacion de IA	Contraste con fuente oficial
MongoDB: embedding permite consultar datos relacionados en una operacion y es adecuado si los hijos se acceden en contexto del padre.	Respaldada por MongoDB Manual, Embedded Data Models: describe lectura relacionada en una operacion y recomienda embedding para relaciones uno-a-muchos consultadas en contexto.
PostgreSQL puede mantener reglas de integridad con restricciones declarativas.	Respaldada por PostgreSQL Documentation, Constraints: CHECK, UNIQUE, claves primarias y foraneas permiten imponer reglas de valores y relaciones.

Conclusion (menos de 200 palabras)

Conviene mantener PostgreSQL como fuente autoritativa de pedidos. El modelo relaciona pedidos, clientes, productos y detalles; claves foraneas y restricciones CHECK/UNIQUE ayudan a conservar coherencia de cantidades y precios. MongoDB permite reunir cliente y lineas en un documento, simplificando la lectura frecuente de pedidos completos, y ofrece atomicidad al actualizar ese documento. Sin embargo, duplicar datos entre motores agrega sincronizacion y riesgo de divergencia. En esta prueba, las consultas de estado, producto y total devolvieron los mismos IDs para los cinco pedidos reales seleccionados. Recomiendo PostgreSQL como sistema de registro y MongoDB como proyeccion de lectura solo si una necesidad medida justifica el costo de sincronizacion. Se deben mantener los precios historicos y anonimizar datos personales en las entregas.

Fuentes oficiales

MongoDB Manual, Embedded Data Models: https://www.mongodb.com/docs/manual/data-modeling/embedding/

PostgreSQL Documentation, Constraints: https://www.postgresql.org/docs/current/ddl-constraints.html

Archivos de entrega

consultas_mongodb.js; consultas_postgresql.sql; evidencias/evidencia_mongodb.txt; evidencias/evidencia_postgresql.txt; evidencias/comparacion_integridad.txt.